```
Start coding or generate with AI.
```

Task

Analyze the `renew_messy.csv` file by loading it into a pandas DataFrame, performing exploratory data analysis, preprocessing the data, training and evaluating Linear Regression, Ridge Regression, Lasso Regression, and Elastic Net Regression models, comparing their performance, and summarizing the findings.

Load Data

Subtask:

Load the 'renew_messy.csv' file into a pandas DataFrame.

Reasoning: The subtask requires loading a CSV file into a pandas DataFrame and displaying its first few rows. This code block will import pandas, read the specified CSV file, and then show the head of the DataFrame.

```
import pandas as pd

df = pd.read_csv('/content/renew_messy.csv')
df.head()
```

	State	Year	Installed - Bio-Mass Power	Installed - Small Hydro Power	Installed - Solar Power	Installed - Wind Power	Zone	Installed - Waste to Energy	Popul
0	Uttar Pradesh	2016.740706	2190.441518	32.516925	936.300954	47.646306	North	-1.183051	217750416.06
1	Andhra Pradesh	2020.512531	597.429912	113.402831	4279.320873	4341.881957	South	82.562010	53113907.6526
2	Arunachal Pradesh	2022.359018	12.836554	107.930798	-268.216550	-306.262502	North-East	0.403803	3797569.9385
3	Chandigarh	2018.329070	-9.571122	14.160004	-216.045221	143.355467	North	-1.152033	
4	Andhra Pradesh	2013.420154	432.157101	193.530181	-13.054656	730.113304	South	-1.245357	50626682.069

Next steps: Generate code with df New interactive sheet

Exploratory Data Analysis (EDA)

Subtask:

Perform initial EDA: check data types, missing values, descriptive statistics, and visualize distributions.

Reasoning: First, I'll display the data types, check for missing values, and generate descriptive statistics for the DataFrame. These are fundamental steps in EDA and provide a quick overview of the dataset's structure and content.

```
print("DataFrame Info:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDescriptive Statistics:")
df.describe()
```

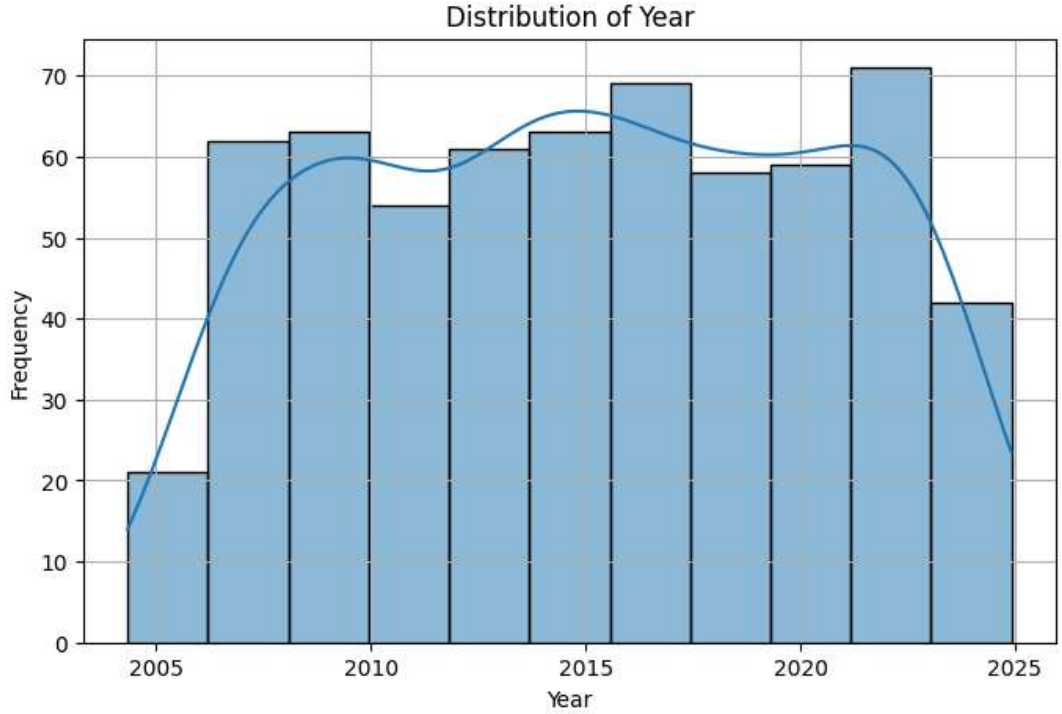
```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 655 entries, 0 to 654
Data columns (total 10 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   State                         623 non-null    object
 1   Year                          623 non-null    float64
 2   Installed - Bio-Mass Power    621 non-null    float64
 3   Installed - Small Hydro Power 623 non-null    float64
 4   Installed - Solar Power       623 non-null    float64
 5   Installed - Wind Power        623 non-null    float64
 6   Zone                          623 non-null    object
 7   Installed - Waste to Energy   622 non-null    float64
 8   Population                    622 non-null    object
 9   GSDP (crore INR)              622 non-null    float64
dtypes: float64(7), object(3)
memory usage: 51.3+ KB

Missing Values:
State                           32
Year                            32
Installed - Bio-Mass Power      34
Installed - Small Hydro Power   32
Installed - Solar Power         32
Installed - Wind Power          32
Zone                            32
Installed - Waste to Energy     33
Population                      33
GSDP (crore INR)                33
dtype: int64

Descriptive Statistics:
```

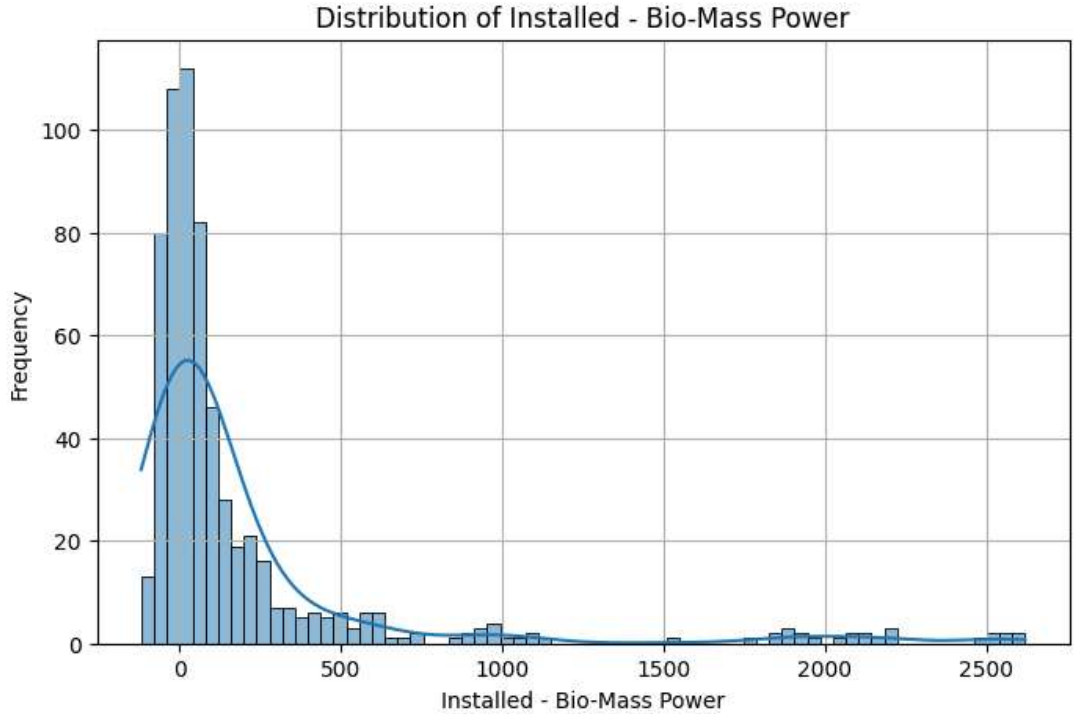
	Year	Installed - Bio-Mass Power	Installed - Small Hydro Power	Installed - Solar Power	Installed - Wind Power	Installed - Waste to Energy	GSDP (crore INR)
count	623.000000	621.000000	623.000000	623.000000	623.000000	622.000000	6.220000e+02
mean	2015.051838	187.098005	115.284653	751.937654	773.323451	3.582900	3.097911e+05
std	5.474425	460.165642	217.530295	2464.834941	1957.462645	12.152515	3.916606e+05
min	2004.330126	-117.975888	-45.559382	-548.351016	-497.895769	-3.465603	-8.269334e+04
25%	2010.219875	-15.015660	15.562805	-118.448649	-106.946490	-0.686675	3.812790e+04
50%	2015.047068	40.020668	54.026475	112.091462	56.437617	0.310682	1.622877e+05
75%	2019.583535	154.365391	120.938279	366.697533	264.331250	1.330944	4.681439e+05
max	2024.917020	2619.804751	1299.592714	25759.905341	11999.092583	82.562010	1.999987e+06

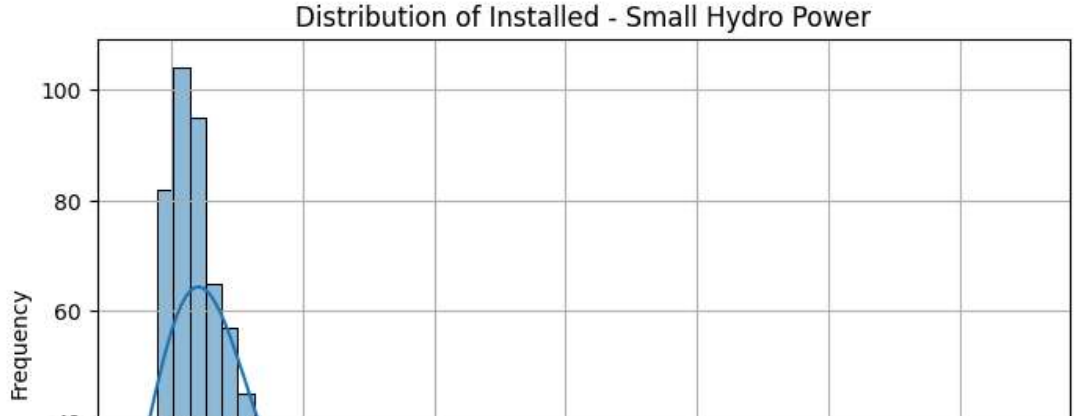
Reasoning: To visualize the distributions of numerical features, I will create histograms using `matplotlib` and `seaborn`. Then, for categorical features, I will display value counts to understand their unique values and frequencies, as requested in the instructions.

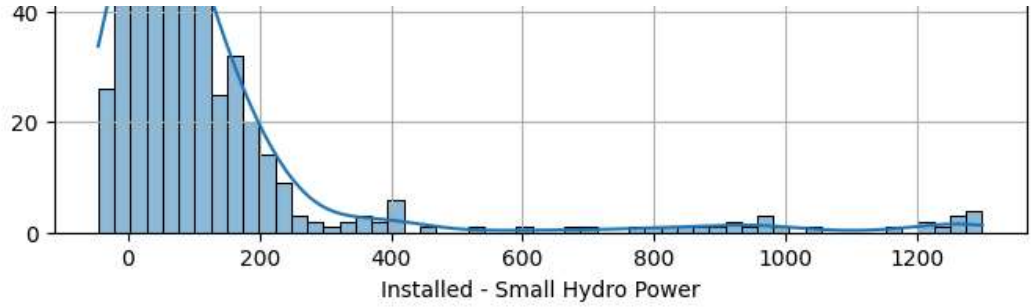
```
import matplotlib.pyplot as plt
import seaborn as sns

# Convert 'Population' to numeric, coercing errors to NaN
df['Population'] = pd.to_numeric(df['Population'], errors='coerce')

# Identify numerical and categorical columns
numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns
categorical_cols = df.select_dtypes(include=['object']).columns

print("\n--- Visualizing Numerical Columns (Histograms) ---")
for col in numerical_cols:
    plt.figure(figsize=(8, 5))
    sns.histplot(df[col].dropna(), kde=True)
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.grid(True)
    plt.show()

print("\n--- Value Counts for Categorical Columns ---")
for col in categorical_cols:
    print(f"\nValue counts for '{col}':")
    print(df[col].value_counts(dropna=False))
```

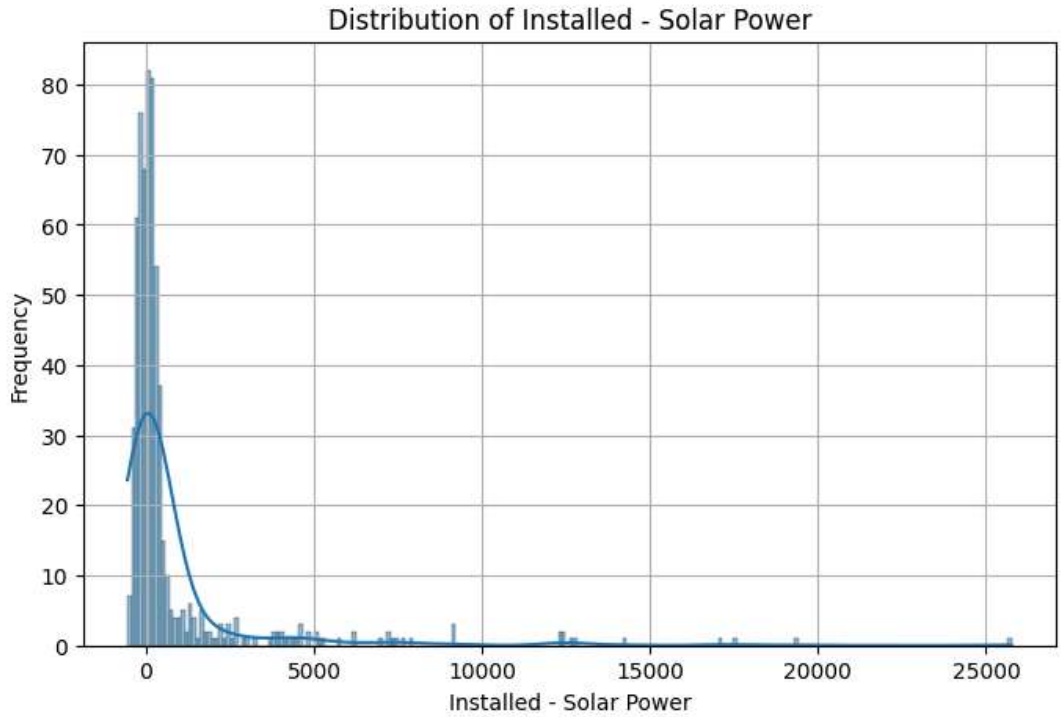
```
--- Visualizing Numerical Columns (Histograms) ---
```

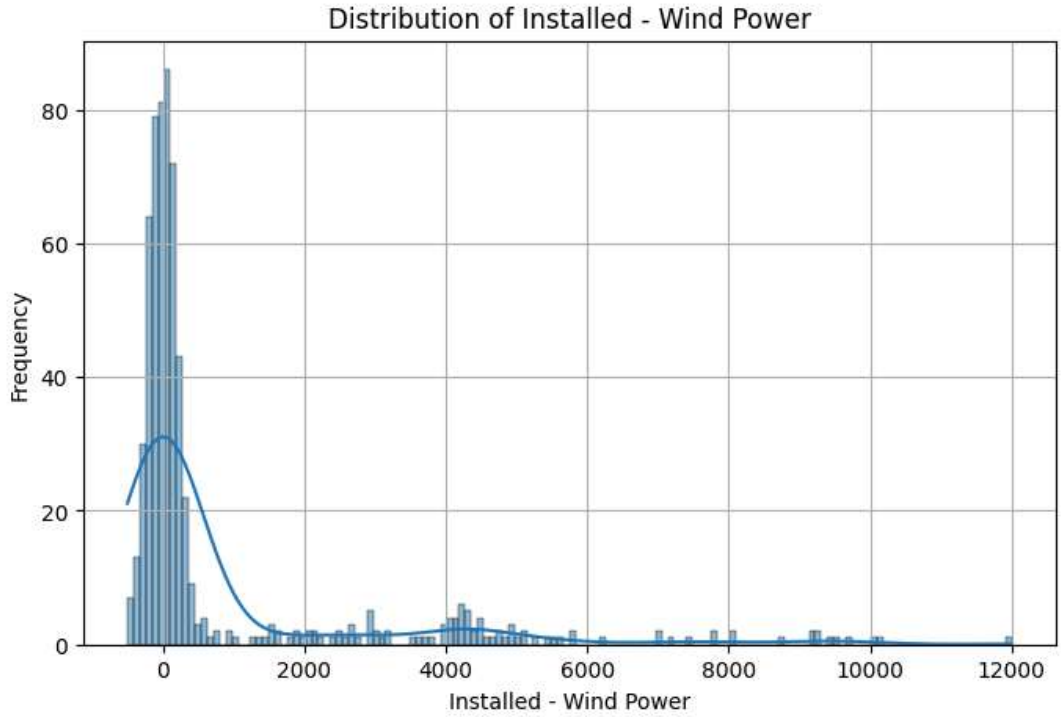


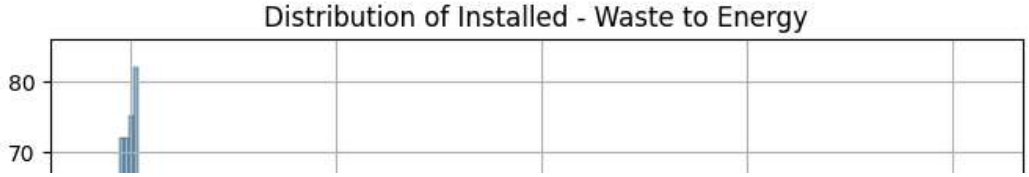


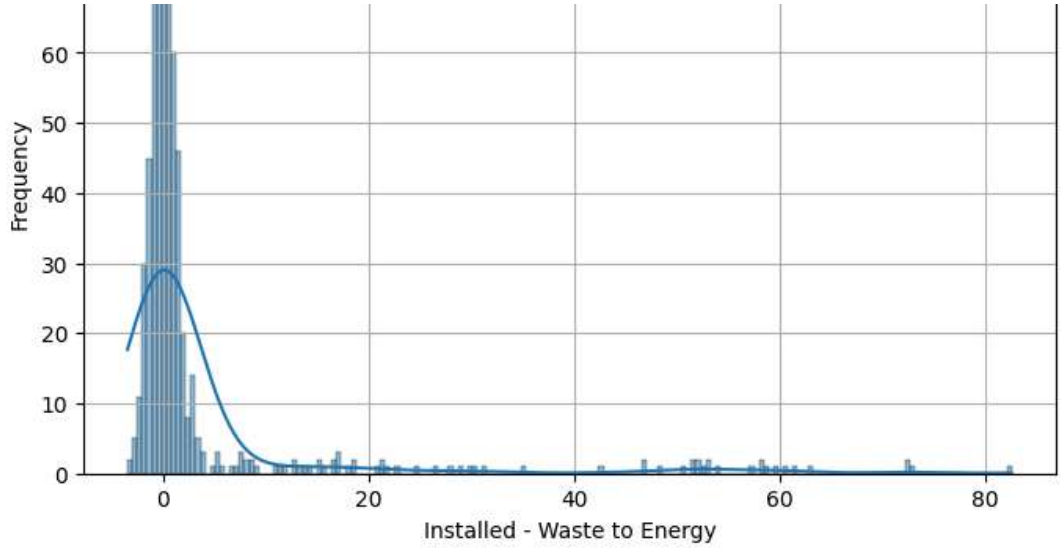


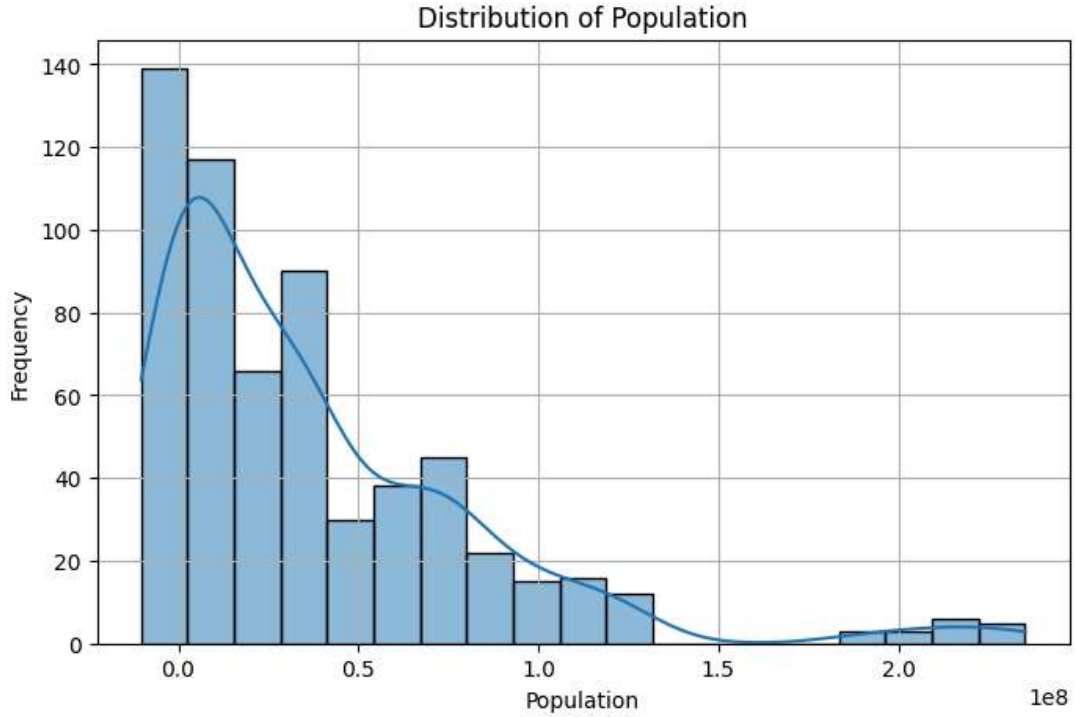

40
20
0
0
200
400
600
800
1000
1200
Installed - Small Hydro Power

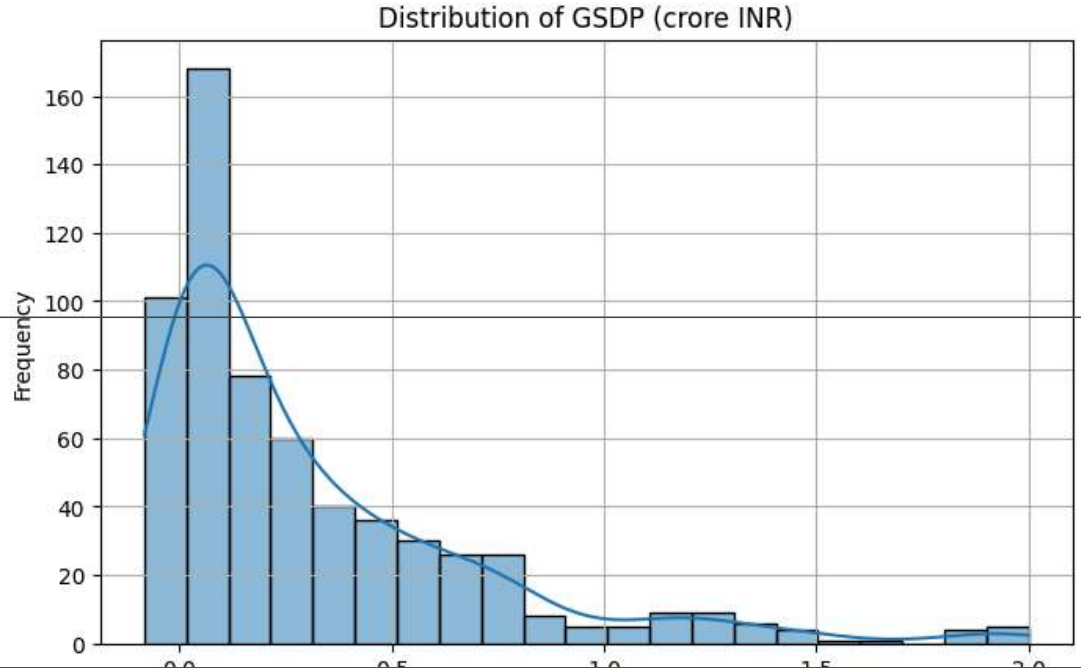

Distribution of Installed - Solar Power
Frequency
80
70
60
50
40
30
20
10
0
0
5000
10000
15000
20000
25000
Installed - Solar Power

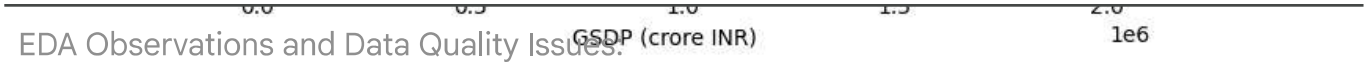

Distribution of Installed - Wind Power
Frequency
80
60
40
20
0
0
2000
4000
6000
8000
10000
12000
Installed - Wind Power


Distribution of Installed - Waste to Energy
80
70


Frequency
Installed - Waste to Energy


Distribution of Population
Frequency
Population
1e8


Distribution of GSDP (crore INR)
Frequency



EDA Observations and Data Quality Issues:

```
--- Value Counts for Categorical Columns ---
Value counts for 'State':
State
Puducherry                    32
Punjab                        21
Tripura                       21
Rajasthan                     20
Uttar Pradesh                 20
Mizoram                       20
Andaman And Nicobar Islands   20
Nagaland                      20
Kerala                        19
Sikkim                        19
Jharkhand                     19
Jammu And Kashmir             19
Goa                           19
Manipur                       19
Gujarat                       19
Karnataka                     19
Bihar                         19
Uttarakhand                   19
Arunachal Pradesh             18
Andhra Pradesh                18
Odisha                        18
Lakshadweep                   18
Haryana                       18
Tamil Nadu                    18
West Bengal                   18
Maharashtra                   18
Madhya Pradesh                18
Himachal Pradesh              17
Telangana                     17
Meghalaya                     16
Assam                         16
Delhi                         16
Chhattisgarh                  16
Chandigarh                    12
Name: count, dtype: int64
Value counts for 'Zone':
Zone
North-East    154
South         148
North         139
East          72
West          72
Central       38
NaN           32
Name: count, dtype: int64
```

1. **Data Types**: Most columns are appropriately typed, with numerical features as `float64` and categorical features as `object`. However, the 'Population' column was initially `object` type and has been converted to `float64` with errors coerced to NaN, indicating potential non-numeric entries.
2. **Missing Values**: There are a significant number of missing values across almost all columns, ranging from 32 to 34 entries. This indicates that imputation or removal of rows with missing values will be necessary during preprocessing.
3. **Descriptive Statistics**:
   - **Year**: Ranges from 2004.33 to 2024.91, suggesting these might not be exact calendar years but rather some form of aggregated or interpolated time points.
   - **Installed Power Columns**: Many 'Installed Power' columns (`Installed - Bio-Mass Power`, `Installed - Small Hydro Power`, `Installed - Solar Power`, `Installed - Wind Power`, `Installed - Waste to Energy`) show **negative minimum values**. Installed power capacity should inherently be non-negative. These negative values are highly unusual and represent a significant data quality issue that needs to be addressed (e.g., as erroneous entries, or representing changes that could be negative, but 'installed' capacity usually implies a positive, cumulative number).
   - **GSDP (crore INR)**: Also shows a negative minimum value, which is highly unlikely for Gross State Domestic Product. This is another critical data quality concern.
4. **Histograms**: The distributions of most numerical 'Installed Power' columns are highly skewed to the right, indicating a few states with very high installed capacities. The presence of negative values is also visible in the left tails of some histograms, reinforcing the descriptive statistics findings.
5. **Categorical Columns**:
   - **State**: Features 35 unique states/territories, with varying counts. There are 32 missing values for 'State'.
   - **Zone**: Features 6 unique zones, with varying counts. There are 32 missing values for 'Zone'.

Summary of Data Quality Issues: The most critical data quality issues identified are the presence of negative values in `Installed Power` columns, `GSDP (crore INR)`, and the 'Population' column (after conversion, potential NaNs from non-numeric entries). Additionally, a substantial number of missing values across the dataset will require careful handling.

Data Preprocessing

Subtask:

Prepare the data for modeling: handle categorical variables, scale features, and split into training and testing sets.

Reasoning: The first step in data preprocessing is to address illogical negative values. I will identify the specified columns and replace any negative values within them with NaN, making them available for subsequent imputation.

```
import numpy as np

# Identify columns that should not logically contain negative values
il_numeric_cols = [
    'Installed - Bio-Mass Power',
    'Installed - Small Hydro Power',
    'Installed - Solar Power',
    'Installed - Wind Power',
    'Installed - Waste to Energy',
```

```
    'GSDP (crore INR)'
]

# Replace negative values with NaN in the identified columns
for col in il_numeric_cols:
    df.loc[df[col] < 0, col] = np.nan

print("Negative values in specified columns replaced with NaN.")
print("Updated missing values count:")
print(df[il_numeric_cols].isnull().sum())
```

```
Negative values in specified columns replaced with NaN.
Updated missing values count:
Installed - Bio-Mass Power       232
Installed - Small Hydro Power    124
Installed - Solar Power          266
Installed - Wind Power           282
Installed - Waste to Energy      295
GSDP (crore INR)                  99
dtype: int64
```

Reasoning: Now that negative values have been converted to NaN, I will impute the missing values. Numerical columns will be imputed with their median, and categorical columns with their mode, as specified in the instructions. This is crucial for handling the significant number of missing values identified during EDA.

```
from sklearn.impute import SimpleImputer

# Identify numerical and categorical columns for imputation
numerical_cols_for_imputation = df.select_dtypes(include=['float64', 'int64']).columns
categorical_cols_for_imputation = df.select_dtypes(include=['object']).columns

# Impute numerical columns with the median
median_imputer = SimpleImputer(strategy='median')
for col in numerical_cols_for_imputation:
    if col in df.columns:
        df[col] = median_imputer.fit_transform(df[[col]]).ravel()

# Impute categorical columns with the mode
mode_imputer = SimpleImputer(strategy='most_frequent')
for col in categorical_cols_for_imputation:
    if col in df.columns:
        df[col] = mode_imputer.fit_transform(df[[col]]).ravel()

print("Missing values imputed for numerical (median) and categorical (mode) columns.")
print("Updated missing values count:")
print(df.isnull().sum())
```

```
Missing values imputed for numerical (median) and categorical (mode) columns.
Updated missing values count:
State                            0
Year                             0
Installed - Bio-Mass Power       0
Installed - Small Hydro Power    0
Installed - Solar Power          0
Installed - Wind Power           0
Zone                             0
Installed - Waste to Energy      0
Population                       0
GSDP (crore INR)                 0
dtype: int64
```

Reasoning: Now that missing values have been handled, I will apply one-hot encoding to the categorical columns ('State', 'Zone') to convert them into a format suitable for machine learning models, as specified in the instructions.

```
import pandas as pd

# Apply one-hot encoding to categorical columns
df = pd.get_dummies(df, columns=['State', 'Zone'], drop_first=True)

print("Categorical columns one-hot encoded.")
print("DataFrame head after one-hot encoding:")
print(df.head())
```

```
Categorical columns one-hot encoded.
DataFrame head after one-hot encoding:
          Year  Installed - Bio-Mass Power  Installed - Small Hydro Power  \
0  2016.740706                 2190.441518                      32.516925
1  2020.512531                  597.429912                     113.402831
2  2022.359018                   12.836554                     107.930798
3  2018.329070                   86.636223                      14.160004
4  2013.420154                  432.157101                     193.530181

   Installed - Solar Power  Installed - Wind Power  \
0              936.300954               47.646306
1             4279.320873             4341.881957
2              270.621574              203.638274
3              270.621574              143.355467
4              270.621574              730.113304

   Installed - Waste to Energy    Population  GSDP (crore INR)  \
0                     1.220758  2.177504e+08      1.104191e+06
1                    82.562010  5.311391e+07      7.290053e+05
2                     0.403803  3.797570e+06      3.871109e+04
3                     1.220758  2.534497e+07      3.222453e+04
4                     1.220758  5.062668e+07      5.149294e+05

   State_Andhra Pradesh  State_Arunachal Pradesh  ...  State_Telangana  \
0                 False                    False  ...            False
1                  True                    False  ...            False
2                 False                     True  ...            False
3                 False                    False  ...            False
4                  True                    False  ...            False

   State_Tripura  State_Uttar Pradesh  State_Uttarakhand  State_West Bengal  \
0          False                 True              False              False
1          False                False              False              False
2          False                False              False              False
3          False                False              False              False
4          False                False              False              False

   Zone_East  Zone_North  Zone_North-East  Zone_South  Zone_West
0      False        True            False       False      False
1      False       False            False        True      False
2      False       False             True       False      False
3      False        True            False       False      False
4      False       False            False        True      False

[5 rows x 46 columns]
```

Reasoning: Now that one-hot encoding is complete, I will define the target variable `y` as 'GSDP (crore INR)' and the features `X` as all other columns. Then, I will split the data into training and testing sets, and finally, scale the numerical features using `StandardScaler` to prepare the data for model training, as instructed.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Define the target variable y and features X
y = df['GSDP (crore INR)']
X = df.drop('GSDP (crore INR)', axis=1)

# Split the data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

# Identify numerical columns for scaling (excluding one-hot encoded columns)
# One-hot encoded columns are already binary (0 or 1) and don't need scaling in this context
# and can be identified by their 'State_' or 'Zone_' prefixes, or by checking dtypes

# Get numerical columns from the original (non-one-hot encoded) df that are not target
original_numerical_cols = ['Year', 'Installed - Bio-Mass Power', 'Installed - Small Hydro Power',
                           'Installed - Solar Power', 'Installed - Wind Power', 'Installed - Waste to Ener

# Filter these columns to ensure they exist in X_train (after dropping 'GSDP (crore INR)')
scaling_cols = [col for col in original_numerical_cols if col in X_train.columns]

# Initialize StandardScaler
scaler = StandardScaler()

# Fit on training data and transform both training and testing data
X_train[scaling_cols] = scaler.fit_transform(X_train[scaling_cols])
X_test[scaling_cols] = scaler.transform(X_test[scaling_cols])

print("Features scaled using StandardScaler.")
print("X_train head after scaling:")
print(X_train.head())
```

```
X_train shape: (524, 45)
X_test shape: (131, 45)
y_train shape: (524,)
y_test shape: (131,)
Features scaled using StandardScaler.
X_train head after scaling:
         Year  Installed - Bio-Mass Power  Installed - Small Hydro Power  \
321 -0.567364                   -0.322153                      -0.267327
456 -1.365638                   -0.322153                      -0.470530
340 -0.034721                   -0.322153                      -0.485167
29  -1.250598                   -0.322153                      -0.493313
571 -0.140798                   -0.181913                       0.396011

     Installed - Solar Power  Installed - Wind Power  \
321                -0.266139               -0.474464
456                -0.266139               -0.367462
340                -0.266139               -0.367462
29                 -0.266139               -0.367462
571                -0.209410               -0.411274

     Installed - Waste to Energy  Population  State_Andhra Pradesh  \
321                    -0.346353   -0.785295                 False
456                    -0.275442   -0.755485                 False
340                    -0.275442   -0.712766                 False
29                     -0.275442   -0.693123                 False
571                    -0.268369   -0.335325                 False

     State_Arunachal Pradesh  State_Assam  ...  State_Telangana  \
321                    False        False  ...            False
456                    False        False  ...            False
340                    False        False  ...            False
29                     False        False  ...            False
571                    False        False  ...            False

     State_Tripura  State_Uttar Pradesh  State_Uttarakhand  State_West Bengal  \
321          False                False              False              False
456          False                False              False              False
340          False                False              False              False
29           False                False              False              False
```

```
571          False                False              False              False

     Zone_East  Zone_North  Zone_North-East  Zone_South  Zone_West
321      False       False             True       False      False
456      False       False             True       False      False
340      False       False             True       False      False
29       False       False            False        True      False
571      False        True            False       False      False

[5 rows x 45 columns]
```

Linear Regression Model

Subtask:

Train and evaluate a standard Linear Regression model.

Reasoning: I will train a Linear Regression model, make predictions, and evaluate its performance using R-squared and Mean Squared Error as specified in the instructions. This will involve importing necessary modules, instantiating and training the model, making predictions, and calculating the metrics.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

# Instantiate a Linear Regression model
linear_model = LinearRegression()

# Train the model
linear_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_linear = linear_model.predict(X_test)

# Calculate evaluation metrics
r2_linear = r2_score(y_test, y_pred_linear)
mse_linear = mean_squared_error(y_test, y_pred_linear)

print("--- Linear Regression Model Performance ---")
print(f"R-squared: {r2_linear:.4f}")
print(f"Mean Squared Error: {mse_linear:.2f}")
```

```
--- Linear Regression Model Performance ---
R-squared: 0.6953
Mean Squared Error: 28324969763.07
```

Ridge Regression Model

Subtask:

Train and evaluate a Ridge Regression model.

Reasoning: I will train a Ridge Regression model, make predictions, and evaluate its performance using R-squared and Mean Squared Error as specified in the instructions. This will involve importing the necessary module, instantiating and training the model, making predictions, and calculating the metrics.

```
from sklearn.linear_model import Ridge

# Instantiate a Ridge Regression model
ridge_model = Ridge(alpha=1.0, random_state=42)
```